

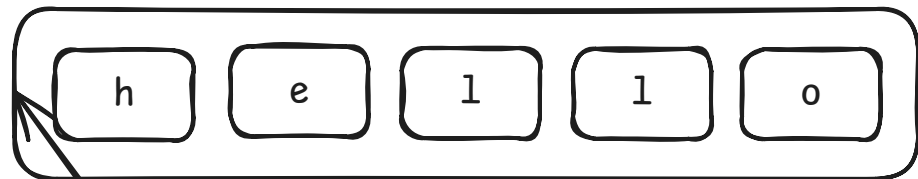
Lecture03 - Strings and Maps

```
s := "hello"
```

```
fmt.Println(len(s)) // 5
```

```
s := "Привет"
```

```
fmt.Println(len(s)) // 12
```



```
type _string struct {  
    d [n]byte  
    len int  
}
```

string
byte
rune

uint8

0: ..
1: ..
..
65: A
66: B
..
255

1 byte

0 - 255

```
var b byte = 65
```

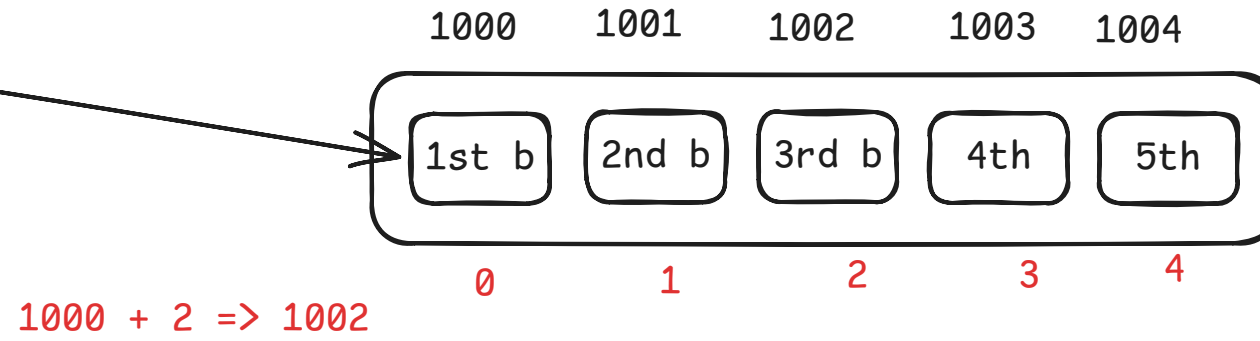
```
fmt.Println(b) // 65  
fmt.Printf("%c", b) // A
```

character

ASCII characters fit into one byte

```
s := "hello"
```

```
fmt.Println(s[2]) // 104  
fmt.Printf("%c", s[0]) // h
```

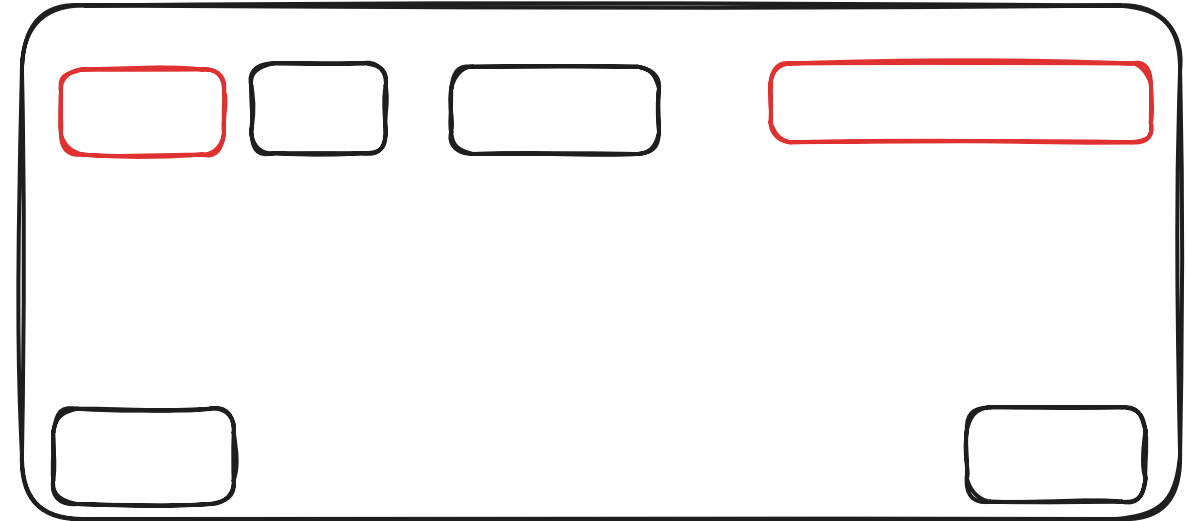


```
s := "hello" -> ASCII : 104 101 ...  
s[0] = 'H' // error
```

```
s := "hello"  
b := []byte(s)
```

```
b[0] = 'H'
```

```
fmt.Println(string(b)) // Hello
```



ASCII

A -> 65
B -> 66
C
...

Unicode

Я -> U+0...
¥ -> U+4F..
θ -> U+....
...

— code point —> 1_... — in Go —> rune

rune — alias —> int32

2^{32}

1 - 4 bytes

0 | 1

----- ... 32

rune —> UTF-8 —> bytes

```
var r rune = 'Я'
```

```
fmt.Println(r) // 1071  
fmt.Printf("%c", r) // Я  
fmt.Printf("%U", r) // U+042F
```

MAPS

```
names := []string{"Alikhan", "Insar", "Malika"}
grades := []int{80, 90, 100}
```

$O(n)$

Insar's grade

Alikhan != Insar

Insar == Insar

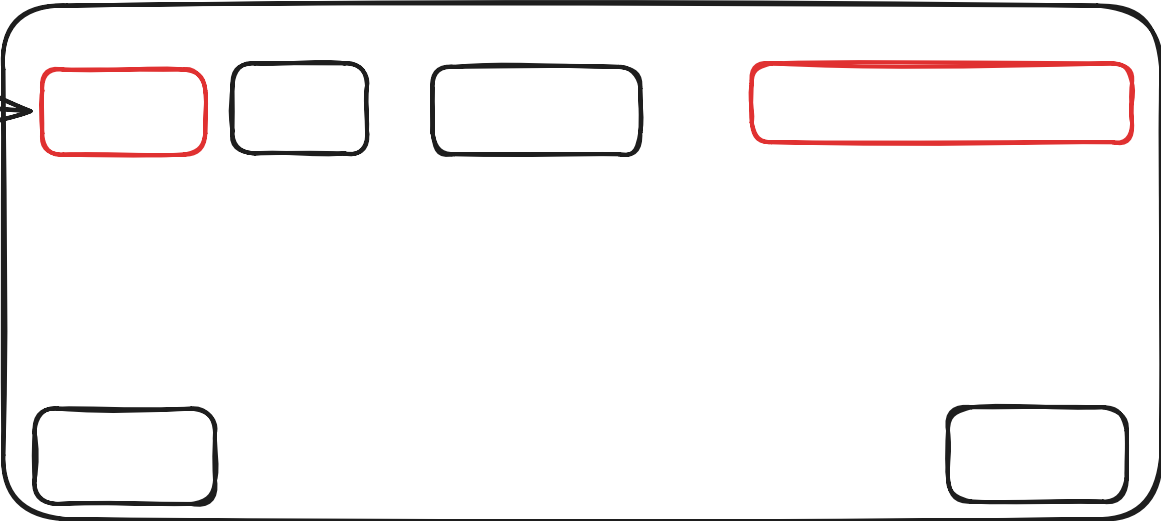
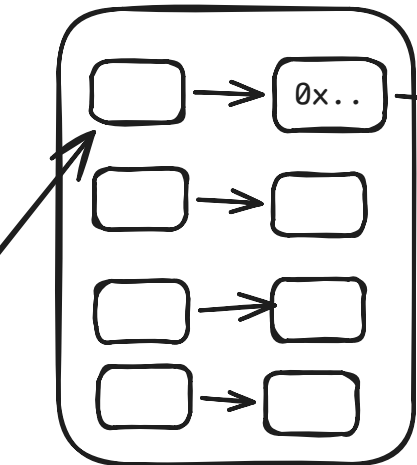
grades[1] -> 90

```
grades := map[string]int{
    "Alikhan": 80,
    "Insar": 90,
    "Malika": 100,
}
```

hash function

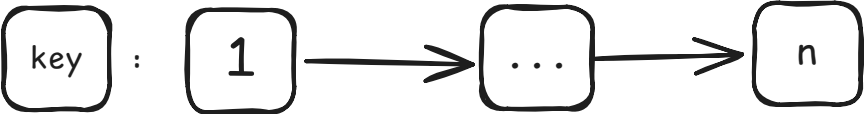
func("Alikhan") -> sgds1Jqr11J151kdfHsa

grades["Alikhan"]



A -> .
B -> ..
C -> ...
.
.
.
0 ->
1 ->

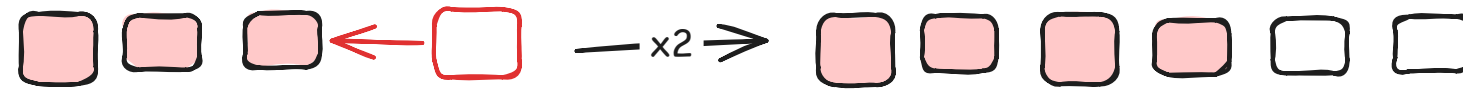
AB -> ...
C -> ... ➔ collision



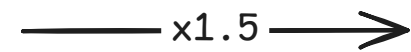
Swiss tables

old version

$3 < 1024$



> 1024



newer versions

some capacity resizing formula

